

POLITECNICO
MILANO 1863

RASD by Hamidreza Khodadadi and
Mohammad Rashidkhan

Requirement Analysis and Specification Document

Deliverable:	RASD
Title:	Requirement Analysis and Verification Document
Authors:	Mohammad Rashid Khan Mofrad and Hamidreza Khodadadi
Version:	1.0
Date:	21-December-2025
Download page:	https://github.com/hamidkhd/KhodadadiRashidkhan/blob/main/README.md
Copyright:	Copyright © 2017, Mohammad Rashid Khan Mofrad and Hamidreza Khodadadi – All rights reserved

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

Contents

Table of Contents **3**

List of Figures **4**

List of Tables **4**

1 Introduction **5**
1.1 Purpose 5
1.2 Scope 6
1.2.1 World Phenomena 6
1.2.2 Shared Phenomena 6
1.3 Definitions, Acronyms, Abbreviations 7
1.3.1 Definitions 7
1.3.2 Acronyms 8
1.3.3 Abbreviations 9
1.4 Revision History 9
1.5 Reference Documents 9
1.6 Document Structure 9

2 Overall Description **11**
2.1 Product Perspective 11
2.1.1 Scenarios 11
2.1.2 Domain-level Class Diagram 12
2.1.3 State Diagrams 14

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

List of Figures

1 Domain class diagram 13
2 Trip State Diagram 14
3 Detected Issue state Diagram 15
4 Path State Diagram 15
5 Obstacle State Diagram Obstacle lifecycle from creation to publication. 16
6 Route query lifecycle from composition to answer. 17

List of Tables

1 Revision history of the RASD. 9

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

1 Introduction

1.1 Purpose

Urban cyclists often find it difficult to identify **safe, comfortable, and efficient routes** in their city. Information about bike lanes, traffic intensity, road quality, and obstacles (such as potholes or damaged surfaces) is usually incomplete, outdated, or scattered across many sources. At the same time, cyclists continuously generate valuable information while riding, such as GPS traces, speed profiles, and movements of their mobile devices, which typically remain unused. As a consequence, planning a pleasant and safe bike trip often relies on personal experience and trial-and-error, which may discourage people from using bikes as a regular means of transportation.

The **Best Bike Paths (BBP)** project aims to address these issues by providing a software system that helps cyclists **collect, maintain, and exploit an inventory of bike paths**. The system supports riders in recording their trips, annotating the quality of the paths they use, and reusing this knowledge to choose suitable routes between an origin and a destination. The ultimate purpose of BBP is to **enable cyclists to plan and perform safer and more effective bike trips** by relying on up-to-date, community-contributed information about bike paths. The purpose of the project is articulated into the following goals:

- **[G1 – Trip Recording]**

  Allow registered users to record their bike trips through a mobile device, capturing GPS traces and timestamps that describe the travelled route.

- **[G2 – Trip Statistics and History]**

  Provide users with statistics for each recorded trip (e.g., distance, duration, average speed) and maintain a personal history of their cycling activities, optionally enriched with relevant meteorological information, when available, from external services.

- **[G3 – Manual Path Definition]**

  Enable registered users to manually define bike paths by selecting streets or segments and associating them with properties such as path status (e.g., *optimal*, *medium*, *sufficient*, *requires maintenance*) and known obstacles (e.g., potholes, hazardous segments).

- **[G4 – Automated Path Data Acquisition]**

  Support an automated mode in which the system, while the user is biking, collects GPS data and sensor readings (accelerometer and gyroscope) to reconstruct the followed path and detect potential issues affecting path quality.

- **[G5 – Confirmation of Inferred Information]**

  Provide mechanisms for users to review, confirm, or correct the information inferred in automated mode (e.g., detected potholes), ensuring that only validated data becomes publishable.

- **[G6 – Path Inventory and Publication]**

  Maintain a structured inventory of bike paths and their associated information, and allow their owners to mark data as publishable so that it becomes available to all users of the system.

- **[G7 – Path Search and Visualization]**

  Allow users, including unregistered users, to request bike paths between a chosen origin and destination and visualize one or more candidate routes on a map.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

- **[G8 – Path Scoring and Recommendation]**

  Compute a score for each candidate path based on its status, obstacles, and effectiveness in reaching the destination from the origin, and use this score to rank alternative paths so that cyclists can easily choose the most suitable one.

Together, these goals define the purpose of BBP: **to turn cyclists' trips and feedback into actionable knowledge that improves the safety and quality of future bike routes.**

1.2 Scope

In the context of the *Best Bike Paths (BBP)* system, the relevant phenomena include:

1.2.1 World Phenomena

- **[WP1]**: Cyclists ride their bikes along existing streets and bike paths in the city, performing trips of varying duration and difficulty.
- **[WP2]**: The physical road infrastructure (streets, bike lanes, intersections) has properties such as surface quality, presence of potholes, and traffic intensity that may change over time.
- **[WP3]**: Weather conditions (e.g., rain, wind, temperature) influence the safety and comfort of cycling on given paths.
- **[WP4]:** Cyclists visually identify the status of the bike path (e.g., optimal, medium, sufficient).
- **[WP5]**: Cyclists plan journeys between an origin and a destination according to their own preferences (e.g., safety, comfort, travel time).
- **[WP6]**: Cyclists gain experience about the quality of the paths they use and may be willing to share this knowledge with others.

1.2.2 Shared Phenomena

World-Controlled Shared Phenomena:

- **[SP1]**: Users register to BBP, authenticate themselves, and manage their personal accounts through the application.
- **[SP2]**: Registered users start and stop the recording of their bike trips using their mobile devices.
- **[SP3]**: Users enable or disable the automated acquisition mode, granting or revoking permission for BBP to use location and motion sensors while they ride.
- **[SP4]**: Registered users manually define bike paths by selecting streets or segments in the application and assigning them a status and known obstacles.
- **[SP5]**: Users review, confirm, or correct information about path issues (e.g., potholes) that BBP has inferred from sensor data before this information becomes publishable.
- **[SP6]**: Users specify an origin and a destination and submit route queries to BBP.
- **[SP7]**: Users inspect the paths displayed by BBP on the map, compare alternatives, and choose which path to follow for their trip.

Machine-Controlled Shared Phenomena:

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

- **[SP8]**: BBP records trip data (GPS positions, timestamps) and stores it as part of the user's trip history.
- **[SP9]**: BBP computes and presents statistics about each recorded trip, such as distance, duration, average speed, and similar indicators, possibly enriched with weather information obtained from external services.
- **[SP10]**: BBP reconstructs the path followed during a trip from GPS data and analyzes motion sensor readings to detect potential obstacles or surface problems.
- **[SP11]:** The system prompts the user to confirm or reject potential obstacles detected during the ride.
- **[SP12]**: BBP updates the inventory of bike paths with manually inserted or confirmed information, including path status and obstacles, and marks data as publishable or private according to the owner's choices.
- **[SP13]**: BBP processes a user's origin–destination query, selects candidate bike paths from the inventory, computes a score for each path based on its quality and effectiveness, and ranks them accordingly.
- **[SP14]**: BBP visualizes the selected paths on a map, together with their scores and relevant details, so that users can evaluate them.
- **[SP15]**: BBP enforces access control and privacy rules when returning trip and path information, exposing only what each user is allowed to see.
- **[SP16]:** The system sends requests to external map/weather APIs to retrieve necessary environmental context.

1.3 Definitions, Acronyms, Abbreviations

This section introduces the terminology used throughout the document. Its aim is to avoid ambiguity and to ensure consistent usage of terms.

1.3.1 Definitions

- **Bike Path:** A route intended to be suitable for bicycles, either because a proper bike lane exists or because traffic conditions (e.g., low car density, compatible speed limits) make the road appropriate for cycling.
- **Path Segment:** A portion of a bike path corresponding, for example, to a single street or a continuous stretch of roadway between two intersections.
- **Trip:** A cycling activity performed by a registered user and recorded by BBP, including the start and end times of the activity and associated statistics such as total distance and average speed.
- **Trip Point (GPS Sample):** A single geolocated point acquired during a trip, consisting of a timestamp and coordinates (latitude and longitude). Trip points form the sequence used by BBP to reconstruct the followed path.
- **Trip Statistics:** Quantitative metrics computed by BBP for a trip, such as total distance, duration, elevation trends, and average speed.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

- **Meteorological Information / Weather Condition:** Weather-related data associated with a trip (e.g., temperature, wind speed, precipitation, general weather conditions) retrieved from an external service.
- **Obstacle:** A relevant issue affecting a bike path, such as a pothole, rough surface, or other road-surface irregularity. Obstacles may be manually reported by users or created automatically after the confirmation of detected issues.
- **Detected Issue:** A potential obstacle inferred automatically by BBP from device sensor data (e.g., accelerometer, gyroscope) acquired during a trip. Each detected issue includes a type, location, and confidence level and must be confirmed, corrected, or rejected by the user.
- **Path Status:** A qualitative evaluation of the overall condition of a bike path (e.g., optimal, medium, sufficient, requires maintenance).
- **Manual Mode:** A mode in which registered users explicitly insert information about bike paths, selecting the streets composing the path and setting their status and known obstacles.
- **Automated Mode:** A mode in which BBP acquires data automatically from the user's mobile device while they are biking. The system infers that the user is biking based on speed, reconstructs the followed path using GPS data, and detects possible obstacles using accelerometer and gyroscope measurements.
- **Confirmation Phase:** A phase following automated acquisition in which the user is asked to confirm, correct, or discard issues inferred by BBP before this information becomes available to the community.
- **Publishable Information:** Information about bike paths provided to BBP (either manually or automatically) that the owner chooses to make available to all users of the system.
- **Registered User:** A user who has an account on BBP and can record trips, insert information about bike paths, confirm automatically detected issues, and publish information.
- **Non-Registered User:** A user who accesses BBP without logging in and can only request the visualization of bike paths between an origin and a destination.
- **Origin:** The starting geographic point of a route specified by the user in a route search query.
- **Destination:** The target geographic point of a route specified by the user in a route search query.
- **Route Query:** A request made by a user to BBP specifying an origin and a destination for which the system must visualize one or more bike paths.
- **Candidate Path:** A bike path evaluated by BBP in response to a route query. For each candidate path, BBP computes a score and displays it as one of the possible routes.
- **Path Score:** A numerical or ordinal value computed by BBP for a bike path based on the path's status, presence of obstacles, and overall effectiveness in reaching the destination from the given origin.

1.3.2 Acronyms

- **BBP**: Best Bike Paths.
- **GPS**: Global Positioning System.
- **RASD**: Requirements Analysis and Specification Document.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

- **UI**: User Interface.
- **API**: Application Programming Interface.

The acronyms UI and API are used in later sections of this document to describe external interfaces and are therefore listed here for completeness.

1.3.3 Abbreviations

- **DB**: Database.
- **ID**: Identifier.
- **Avg.**: Average (e.g., Avg. speed).
- **Min.**: Minimum.
- **Max.**: Maximum.

1.4 Revision History

Version	Date	Author(s)	Description
0.1	2025-11-22	Mohammad Rashidkhan	Prepare Introduction draft and add Purpose and Scope sections.
0.2	2025-11-23	Hamidreza Khodadadi	Add Definitions, Acronyms, Abbreviations, Reference and Structure sections.
0.3	2025-11-28	Mohammad Rashidkhan	Prepare Overall Description draft and add Product Perspective section.

Table 1: Revision history of the RASD.

1.5 Reference Documents

This document is based on the following material:

- The official RASD assignment specification for the Software Engineering 2 course, academic year 2025–2026, containing the problem definition “Best Bike Paths”.
- Slides and notes from the Software Engineering 2 course concerning requirements engineering and the preparation of RASD documents.

1.6 Document Structure

- **Section 1 – Introduction**: presents the purpose and scope of BBP, defines key terminology (definitions, acronyms, abbreviations), and reports revision history, reference documents, and the structure of this RASD.
- **Section 2 – Overall Description**: provides a high-level view of the BBP system, including the product perspective, main functions, user characteristics, assumptions, dependencies, and constraints.
- **Section 3 – Specific Requirements**: details the functional and non-functional requirements of BBP, including external interface requirements, use cases and related diagrams, performance requirements, design constraints, and software system attributes.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

- **Section 4 – Formal Analysis Using Alloy**: presents the formal model of selected aspects of BBP and the results of analyses performed using the Alloy tool.
- **Section 5 – Effort Spent**: summarizes the amount of work carried out by each group member for the preparation of this document.
- **Section 6 – References**: lists all external sources referenced in this RASD.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

2 Overall Description

2.1 Product Perspective

2.1.1 Scenarios

A Cyclist Records a Trip Luca, a regular cyclist who often rides across the city, opens the BBP mobile application before starting his commute. After logging in, he taps the *Start Trip* button. BBP activates Luca's GPS and begins collecting his location data every few seconds. As he rides, the application also monitors the readings from his phone's accelerometer and gyroscope, registering significant vertical or lateral movements that may indicate bumps or potholes on the road.

During the trip, Luca checks the live dashboard, which shows elapsed time, current speed, and total distance covered. At some point, he rides through a rough segment of road; the phone vibrates noticeably, and BBP logs this sensor peak as a potential obstacle.

After reaching his destination, Luca taps *Stop Trip*. The system stores the trip in his personal history and enriches it with weather information (temperature, wind, rain intensity) retrieved from an external weather service. Later, Luca reviews detailed trip statistics such as distance, duration, and average speed. The final GPS position recorded when he stopped the trip is considered the destination of that trip.

A Cyclist Defines a Path Manually Maria wants to contribute to the community by documenting a bike-friendly route she frequently uses to reach her university. She logs into BBP and navigates to the *Manual Path Submission* section.

Here, she is presented with an interactive map. Maria selects the streets she uses, adding each segment to the path. For each selected segment, she assigns a path status such as:

- "Optimal" for recently renewed bike lanes,
- "Medium" for regular asphalt,
- "Sufficient" for a street where traffic is a bit heavier.

Along the route, Maria also marks relevant obstacles: a pothole near a particular intersection, a slippery segment on a bridge when wet, and a construction zone that temporarily narrows one of the lanes. After reviewing her inputs, she marks the path information as *publishable*, making it available to the rest of the BBP community.

A Cyclist Uses Automated Mode to Gather Path Information Alberto enables *Automated Mode* in the BBP app because he enjoys cycling through different parts of the city and wants BBP to automatically gather useful data from his rides. Once activated, the system continuously collects GPS positions, speed, and device motion signals, inferring Alberto's followed path.

During one of his rides, the accelerometer captures several sharp vertical movements when Alberto rides on cobblestones. BBP interprets these readings as possible road surface issues. However, the system knows such detections may include false positives; for example, the cyclist might simply drop the phone or hit a curb.

After the ride, BBP presents Alberto with a list of detected obstacles along with their map locations. For each detection, Alberto can:

- confirm it as a real pothole or bump,
- correct the classification (e.g., "not a pothole, just cobblestones"), or
- reject the detection entirely.

Only after confirmation does BBP add the validated information to the path inventory. Alberto appreciates this workflow because it prevents incorrect data from being published.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

A User Searches for Bike Paths Between an Origin and a Destination Elena, who is planning to ride from her home to a park across town, opens the BBP app without logging in. She goes to the *Find Path* feature and enters her origin and destination. BBP queries its inventory of publishable paths contributed by the community.

The system identifies several potential paths connecting the two points. Each path has an associated score, computed using:

- its overall status,
- the number and severity of obstacles along it,
- and its effectiveness in reaching the destination (for example, length and directness).

BBP then presents Elena with a set of routes on the map. The highest-scoring route is highlighted, while alternatives are also shown. Each route displays a summary of:

- expected distance and time,
- number of obstacles,
- whether segments are optimal, medium, or require maintenance.

Elena selects the highest-scoring route and starts navigating it using the map view provided by the app.

A Cyclist Reviews and Publishes Automatically Detected Information Sara rides daily to work using Automated Mode. After several days, BBP identifies recurring bumps at nearly the same location along her path. The system groups these observations and prompts Sara to validate them.

She opens the *Review Detected Issues* screen, which lists, for each candidate issue:

- location of the issue,
- frequency of detection,
- suggested issue category (e.g., pothole, uneven pavement).

Sara inspects the area on the map and confirms that the issue is indeed a pothole, which has been getting worse over the weeks. She marks the detection as correct and chooses to make the information publishable. The corresponding path segment's status is updated to "requires maintenance", improving BBP's accuracy for other users.

2.1.2 Domain-level Class Diagram

The domain-level class diagram models the key entities and relationships that characterize the Best Bike Paths (BBP) application domain. It represents central concepts such as **Cyclist**, **Trip**, **Path**, **PathSegment**, **Obstacle**, **DetectedIssue**, and **RouteQuery**, along with their relevant attributes and the interactions among them.

The model captures how trips are recorded and reconstructed from GPS samples, how users manually or automatically contribute information about bike paths, how obstacles emerge and become publishable through user confirmation, and how BBP retrieves and ranks candidate paths for route queries. By focusing on world and shared phenomena rather than implementation details, the diagram provides a conceptual overview of how information flows within the domain.

A **Trip** represents a cycling activity recorded by a registered user. Each trip consists of a sequence of **TripPoints**, which are individual GPS samples collected during the ride. After the recording ends,

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

BBP reconstructs the travelled route by geographically matching these TripPoints to the existing **PathSegments** in the inventory. This relationship is represented by the association indicating that a Trip may *traverse* several PathSegments.

During a trip, BBP may infer anomalies from accelerometer and gyroscope data, creating **DetectedIssues**. Each DetectedIssue is associated with exactly one Trip and is localized on exactly one PathSegment, reflecting where the potential obstacle occurred. After the ride, users must review the detected issues: confirmed or corrected ones result in the creation of new **Obstacles** on that segment, while rejected ones are discarded. Obstacles and paths can each be marked as *publishable*, determining the visibility of the information within the shared path inventory.

A **Path** is a route defined by a sequence of PathSegments, either manually specified by the user or derived from previously collected information. Each Path has a qualitative **status** selected from a finite set of values (e.g., optimal, medium, sufficient, requires maintenance). A Path includes at least one PathSegment, while each PathSegment may belong to one or more Paths.

Any user—registered or not—can request suggested bike routes by submitting a **RouteQuery**, specifying an origin and destination, each modeled as a **Location** value object. BBP evaluates candidate paths against the query and constructs **CandidatePath** entities, each referencing a Path and associated with a real-valued score computed from the path's condition, obstacles, and effectiveness in reaching the destination. Non-registered users are not explicitly represented in the diagram; their behavior is captured by RouteQuery instances without an associated Cyclist.

The following diagram provides a conceptual overview of the problem domain and avoids any implementation-specific detail.

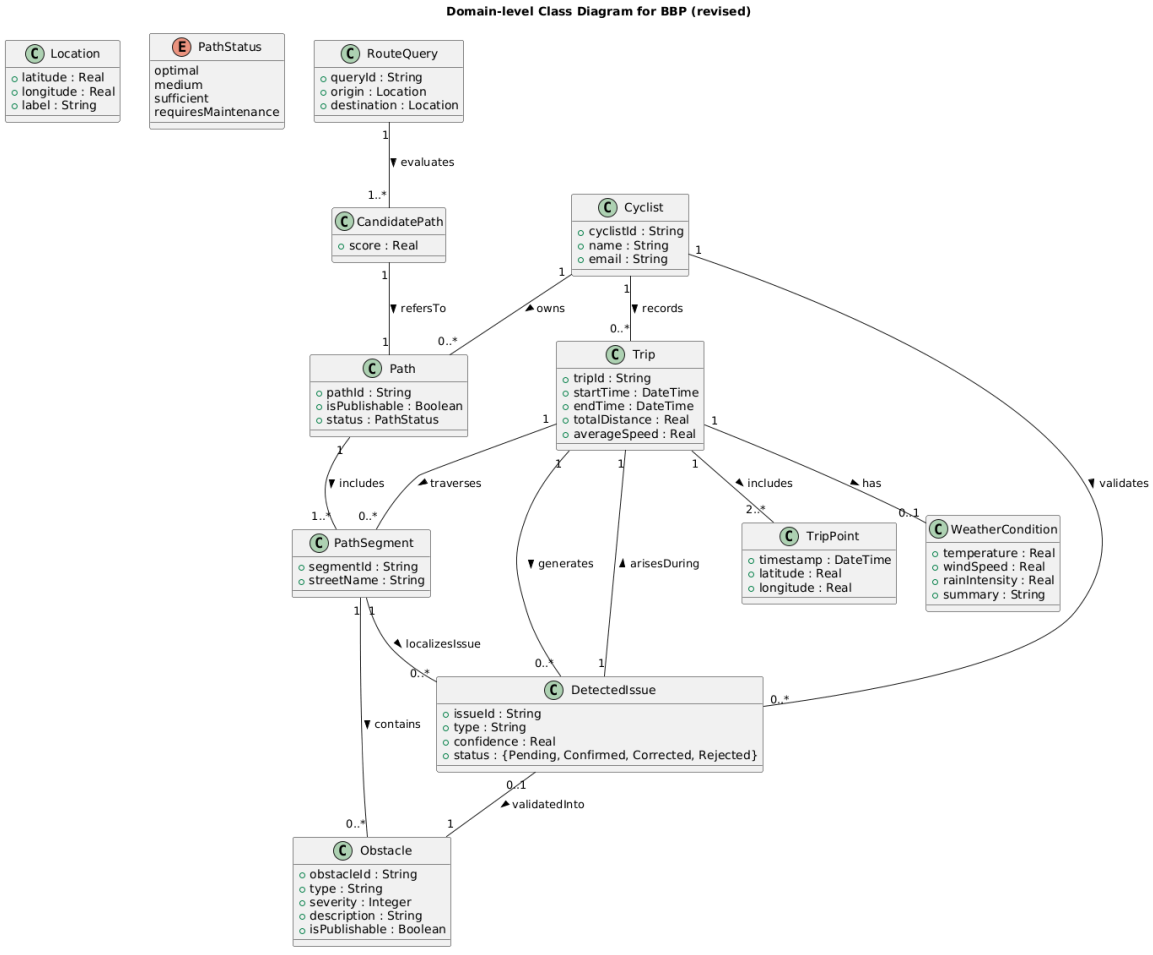


Figure 1: Domain class diagram

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

2.1.3 State Diagrams

State diagrams illustrate the dynamic behavior of key domain entities in BBP. They describe how Trips evolve during recording, how automatically DetectedIssues move through confirmation stages, and how Paths become publishable.

Trip State Diagram
This diagram represents the lifecycle of a recorded Trip. A trip begins when a cyclist starts recording. While the trip is ongoing, BBP collects GPS samples and sensor readings. When the cyclist stops recording, the system reconstructs the travelled route, computes statistics, optionally retrieves weather data, and stores the completed trip in the user's history.

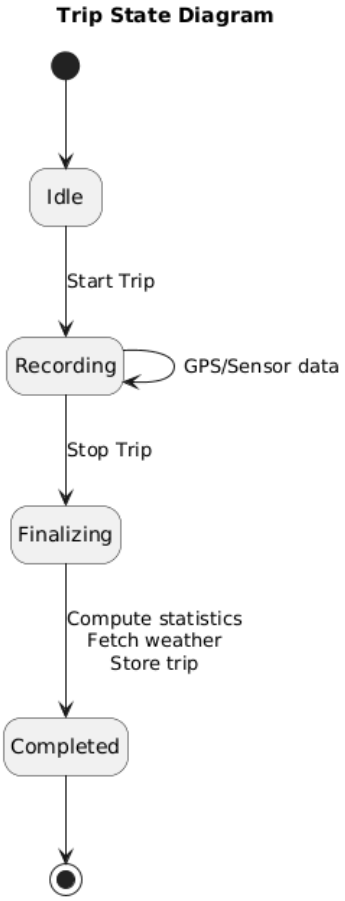


Figure 2: Trip State Diagram

Detected Issue State Diagram
This diagram models the lifecycle of a Detected Issue, that is, a potential obstacle inferred automatically from accelerometer and gyroscope anomalies during a trip. Since the system may produce false positives, each detected issue must go through a confirmation phase where the user validates, corrects, or rejects the inferred information. Confirmed or corrected issues become Obstacles; rejected ones are discarded.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

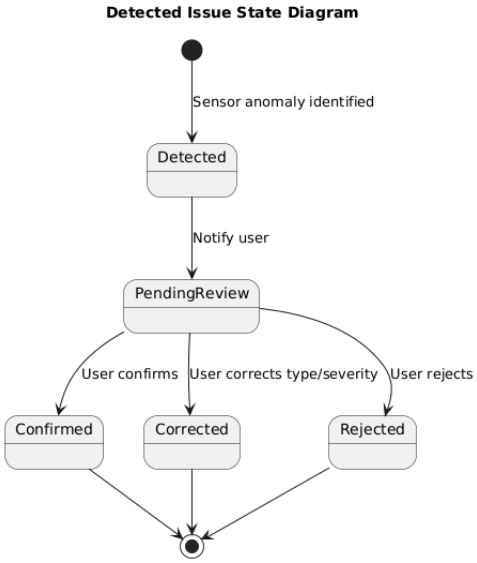


Figure 3: Detected Issue state Diagram

Path State Diagram

This diagram represents the lifecycle of a Path defined by a registered user. A path starts as a draft in manual mode, while the user selects segments and sets status. The path can then be saved as non-publishable (private), updated later, or made publishable so that all users can access it. Users may also update path information over time, leading the path back to the Editable state.

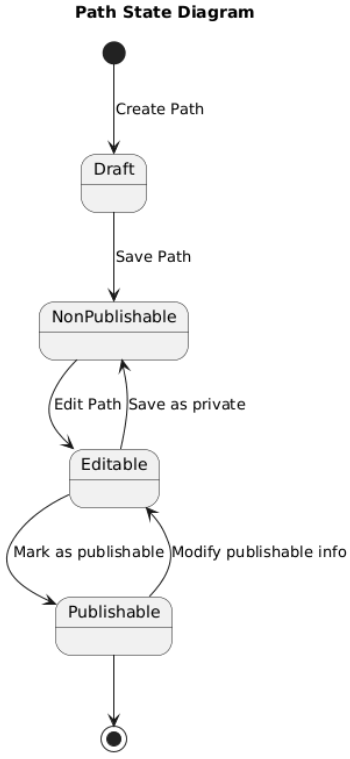


Figure 4: Path State Diagram

Obstacle State Diagram

This diagram represents the lifecycle of an Obstacle, i.e., a confirmed issue associated with a path seg-

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

ment. An obstacle can be created either manually by the user or automatically after the confirmation of a detected issue. When created, the obstacle is initially private to its owner. The owner can then decide whether to publish or unpublish it, controlling its visibility in the shared path inventory.

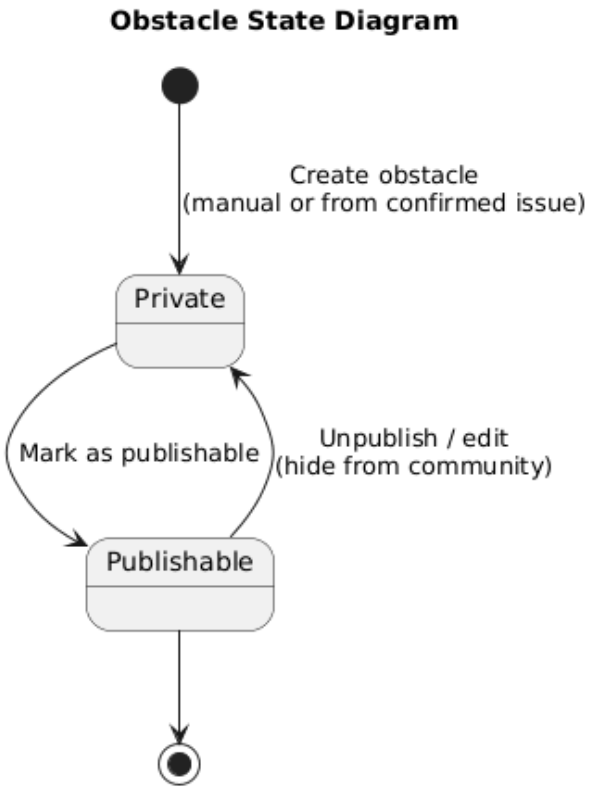


Figure 5: Obstacle State Diagram
Obstacle lifecycle from creation to publication.

RouteQuery State Diagram
This diagram represents the lifecycle of a **RouteQuery**, that is, a user request to visualize bike paths between an origin and a destination. A query is first composed by the user (setting origin and destination), then submitted to BBP for processing. The system retrieves candidate paths, computes their scores, and finally presents the ordered results on the map. After the answers are shown, the query reaches a completed state. Non-registered users issue queries in exactly the same way; they are not modeled explicitly here and are represented simply by RouteQuery instances without an associated Cyclist.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved

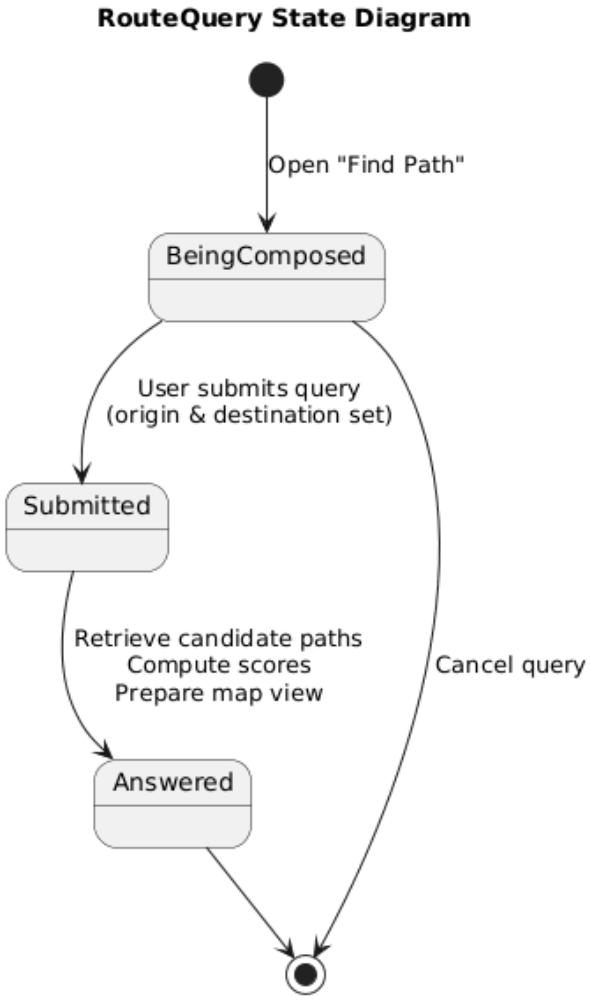


Figure 6: Route query lifecycle from composition to answer.

Copyright © 2025, Hamidreza Khodadadi and Mohammad Rashidkhan – All rights reserved